



Procedures e Funções no PostgreSQL

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



O Problema

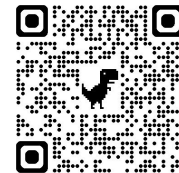
- De volta ao nosso sistema empresarial com empregados, departamentos, projetos e afins...
 - Nosso reaproveitamento das coisas que fizemos é limitado à **SELECTs** e em **VIEWS** que as salvam
 - Porém, outras operações acontecem frequentemente em sistemas como o nosso, e algumas podem modificar dados diretamente no banco por meio de inserções, atualizações e deleções!
- > é preciso reescrever essas operações sempre que precisarmos realizar essas atividades?



Funções

- Permitem amplificar o reuso de código SQL
- Automatizando tarefas e centralizando regras do sistema
- Alguns Exemplos:
 - Cálculo de folha salarial
 - Buscar departamento
 - Buscar dependentes
 - etc.

Funções



```
1 -- Cria uma função chamada dobro_salario
2 -- Recebe um valor do tipo NUMERIC como parâmetro
3 CREATE FUNCTION dobro_salario(valor NUMERIC)
4 -- Define o tipo do retorno (NUMERIC)
5 RETURNS NUMERIC AS
6 -- A função começa aqui ($$)
7 $$
8 BEGIN
9     RETURN valor * 2; -- retorna o dobro
10 END;
11 -- A função acaba aqui ($$)
12 -- Especifica a linguagem utilizada
13 $$ LANGUAGE plpgsql;
14 -- O postgresql dá suporte a múltiplas linguagens
15 -- Procedural Language/PostgreSQL
```



Funções

- Chamamos a função vide `SELECT`, tal qual chamamos as nossas funções de agregação:



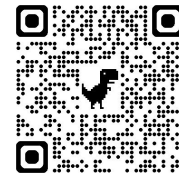
```
1 SELECT dobro_salario(3000);
```



Funções

- Podemos adicionar outros **SELECTS** no corpo das nossas funções, bem como outras funções, como as de agregação.
- Além disso, é possível declarar variáveis para manipular os valores dentro da função (tal qual a maioria das linguagens de programação), neste caso usamos o bloco **DECLARE**.

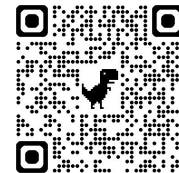
Funções



- Uma função que conta o número de funcionários em um departamento:

```
1 CREATE FUNCTION total_funcionarios(dep_nome VARCHAR)
2 RETURNS INTEGER AS
3 $$
4 DECLARE -- declaração de variáveis
5     total INTEGER;
6 BEGIN
7     SELECT COUNT(*)
8     INTO total -- atribui a saída da consulta à variável total
9     FROM empregado
10    INNER JOIN departamento
11        ON empregado.numero_departamento = departamento.numero
12    WHERE departamento.nome = dep_nome;
13
14    RETURN total;
15 END;
16 $$ LANGUAGE plpgsql;
```

Funções



- Vamos ver no que vai dar a seguinte execução:

```
1 SELECT total_funcionarios('RH');
```




Exercícios

- Crie uma função chamada `departamento_funcionario()` que receba o cpf do funcionário e retorne o nome do departamento desse funcionário.

```
1 CREATE FUNCTION departamento_funcionario(cpf_func VARCHAR)
2 RETURNS VARCHAR AS
3 $$
4 DECLARE
5     nome_dep VARCHAR;
6 BEGIN
7     SELECT departamento.nome
8     INTO nome_dep
9     FROM empregado
10    INNER JOIN departamento
11           ON empregado.numero_departamento = departamento.numero
12    WHERE empregado.cpf = cpf_func;
13
14     RETURN nome_dep;
15 END;
16 $$ LANGUAGE plpgsql;
```



Exercícios

- Crie uma função chamada `maior_salario()` que retorna o maior salário da empresa

```
1 CREATE FUNCTION maior_salario()  
2 RETURNS NUMERIC AS  
3 $$  
4 DECLARE  
5     maior NUMERIC;  
6 BEGIN  
7     SELECT MAX(salario)  
8     INTO maior  
9     FROM empregado;  
10  
11     RETURN maior;  
12 END;  
13 $$ LANGUAGE plpgsql;
```



Exercícios

- Crie uma função chamada `total_projetos_departamento()` que receba o número de um departamento e retorne quantos projetos pertencem a esse departamento

```
1 CREATE FUNCTION total_projetos_departamento(dep_num INTEGER)
2 RETURNS INTEGER AS
3 $$
4 DECLARE
5     total INTEGER;
6 BEGIN
7     SELECT COUNT(DISTINCT projeto.codigo)
8     INTO total
9     FROM departamento
10    INNER JOIN empregado
11        ON empregado.numero_departamento = departamento.numero
12    INNER JOIN empregado_projeto
13        ON empregado.cpf = empregado_projeto.cpf_empregado
14    INNER JOIN projeto
15        ON projeto.codigo = empregado_projeto.codigo_projeto
16    WHERE departamento.numero = dep_num;
17
18    RETURN total;
19 END;
20 $$ LANGUAGE plpgsql;
```



Exercícios

- Crie uma função chamada `funcionario_existe()` que recebe o cpf de um funcionário e retorna TRUE caso ele exista e FALSE caso contrário



```
1 CREATE FUNCTION funcionario_existe(cpf_func VARCHAR)
2 RETURNS BOOLEAN AS
3 $$
4 DECLARE
5     total INTEGER;
6 BEGIN
7     SELECT COUNT(*)
8     INTO total
9     FROM empregado
10    WHERE cpf = cpf_func;
11
12    RETURN total > 0;
13 END;
14 $$ LANGUAGE plpgsql;
```



Exercícios

- Crie uma função chamada `media_departamento()` que recebe o nome do departamento e retorna a média salarial dos funcionários desse departamento.

```
1 CREATE FUNCTION media_departamento(dep_nome VARCHAR)
2 RETURNS NUMERIC AS
3 $$
4 DECLARE
5     media NUMERIC;
6 BEGIN
7     SELECT AVG(empregado.salario)
8     INTO media
9     FROM empregado
10    INNER JOIN departamento
11        ON empregado.numero_departamento = departamento.numero
12   WHERE departamento.nome = dep_nome;
13
14     RETURN media;
15 END;
16 $$ LANGUAGE plpgsql;
```



Procedures (Procedimento)

- Executa operações no banco, podendo alterar dados e conter vários comandos SQL
- Ex:
 - cadastrar funcionário
 - atualizar salários
 - registrar novo projeto



Procedures vs Funções

A procedure altera os dados diretamente no banco

Função	Procedure
retorna um valor	executa ações
chamada via SELECT	chamada com CALL
foco em cálculo	foco em processo
ex: calcular média salarial	ex: atualizar salários



Procedures vs. Funções

```
1 -- Cria uma procedure chamada aumentar_salario
2 CREATE PROCEDURE aumentar_salario(
3     -- Parâmetro: número do departamento
4     numero_dep INTEGER,
5     -- Parâmetro: percentual de aumento
6     percentual NUMERIC
7 )
8 -- Início do corpo da procedure
9 AS
10 $$
11 BEGIN
12     -- Atualiza os salários dos empregados
13     UPDATE empregado
14     -- Novo salário
15     SET salario = salario + (salario * percentual / 100)
16     -- Seleciona apenas empregados do departamento informado
17     WHERE numero_departamento = numero_dep;
18 END;
19 $$
20 LANGUAGE plpgsql;
```




Procedures vs. Funções

Chamando uma Procedure



```
1 CALL aumentar_salario(1, 10);
```



Exercícios

- Crie uma procedure chamada `alterar_departamento()` que receba o cpf do funcionário e o novo departamento, faça sua respectiva atualização

```
1 CREATE PROCEDURE alterar_departamento(  
2     cpf_funcionario INTEGER,  
3     novo_departamento INTEGER  
4 )  
5 AS  
6 $$  
7 BEGIN  
8     UPDATE empregado  
9     SET numero_departamento = novo_departamento  
10    WHERE cpf = cpf_funcionario;  
11 END;  
12 $$ LANGUAGE plpgsql;
```

Exercícios



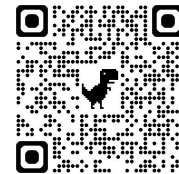
- Crie uma procedure chamada `aumentar_salario_projeto()` que receba o código de um projeto, receba um percentual e aumente o salário dos empregados que participam do projeto

```
1 CREATE PROCEDURE aumentar_salario_projeto(  
2     cod_projeto INTEGER,  
3     percentual NUMERIC  
4 )  
5 AS  
6 $$  
7 BEGIN  
8  
9     UPDATE empregado  
10    SET salario = salario + (salario * percentual / 100)  
11  
12    WHERE cpf IN  
13    (  
14        SELECT cpf_empregado  
15        FROM empregado_projeto  
16        WHERE codigo_projeto = cod_projeto  
17    );  
18  
19 END;  
20 $$ LANGUAGE plpgsql;
```



Exercícios

- Crie uma procedure chamada `cadastrar_departamento()` que receba o nome do departamento e insira um novo departamento na tabela
 - O resto é com você!



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br